南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

计算机组成原理
COMPUTER ORGANIZATION

本课件内容参考自David A. Patterson与John L. Hennessy的教材《Computer Organization and Design: The Hardware/Software Interface》官方配套课件。

zhangj4@sustech.edu.cn

Computer Organization

Lecture 2

RISC-V Instruction Set Architecture

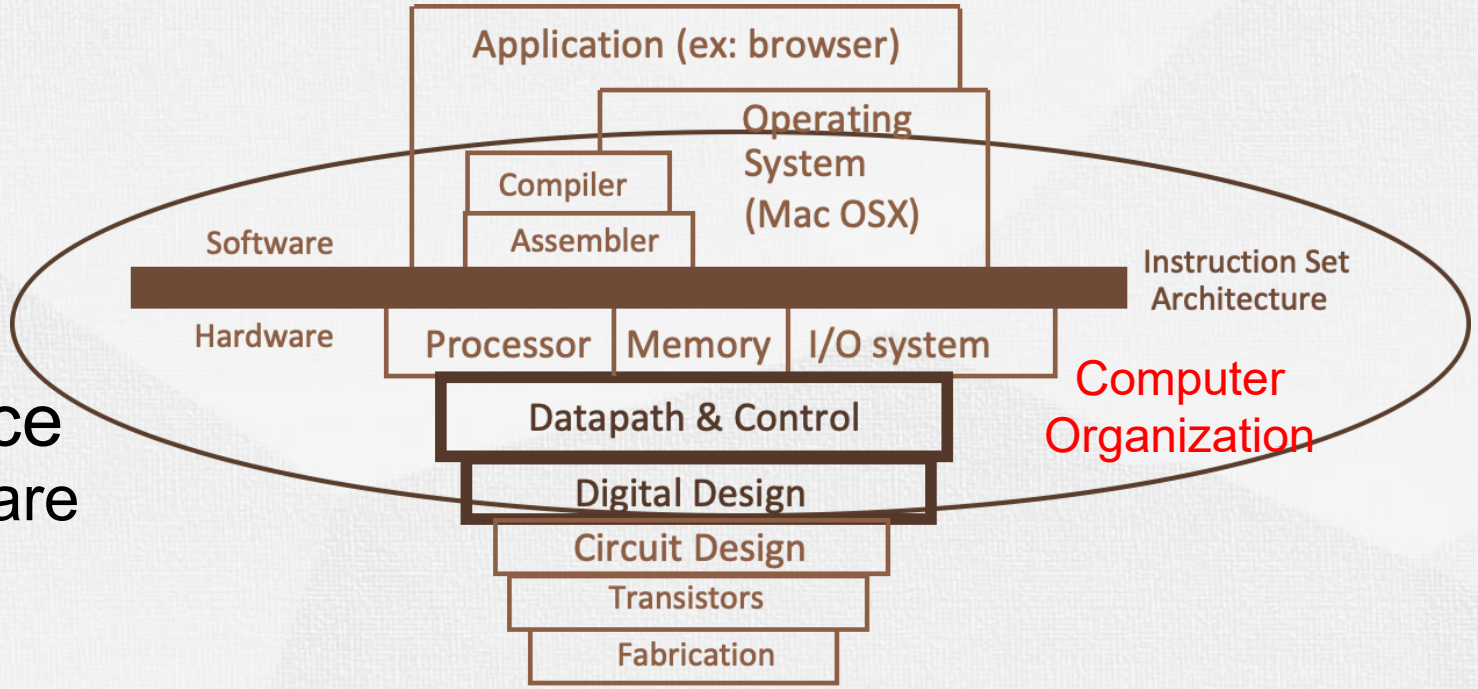
zhangj4@sustech.edu.cn

Outline

- Instruction set architecture
  - RISC vs. CISC
  - RISC-V/MIPS/ARM/x86
- Instructions
  - Arithmetic instruction: add, sub, …
  - Data transfer instruction: lw, sw, lh, sh, …
  - Logical instruction: and, or, …
  - Conditional branch: beq, bne, …
- Basic concepts
  - Operands: register vs. memory vs. immediate
  - Numeric representation: signed, unsigned, sign extension

ReCap: Abstractions

- Abstraction helps us deal with complexity
  - Hide lower-level detail
- **Instruction set architecture (ISA)**
  - The hardware/software interface
- Application binary interface
  - The ISA plus system software interface
- Implementation
  - The details underlying and interface



Instruction Set Architecture (ISA)

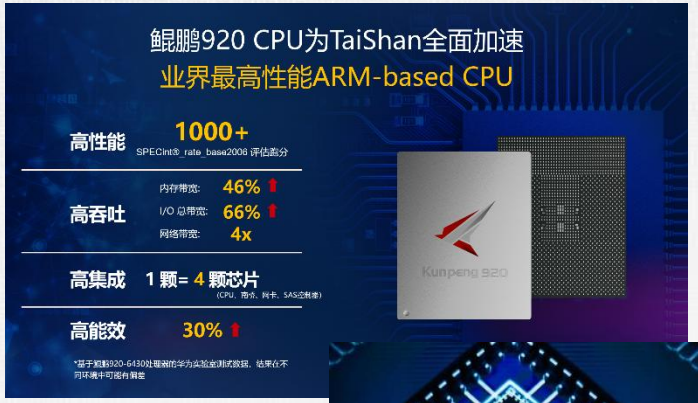
- Instructions: CPU's primitive operations
  - Instructions performed one after another in sequence
  - Each instruction does a small amount of work (a tiny part of a larger program).
  - Each instruction has an operation applied to operands,
  - and might be used to change the sequence of instructions.
- CPUs belong to "families," each implementing its own set of instructions
- CPU's particular set of instructions implements an Instruction Set Architecture (ISA)
  - Examples: ARM, Intel x86, MIPS, RISC-V, PowerPC...

ISA: RISC vs CISC

	RISC (Reduced Instruction Set Computer)	CISC (Complex Instruction Set Computer)
Example	Instruction: Pick up spoon → Scoop rice → Bring to mouth → Swallow	Instruction: Eat meal
Pros & Cons	• Simple, optimized for common operations; can be pipelined efficiently • Multiple instructions needed for one complex task • Execution cycles: ≤ 1 cycle per instruction	• Single instruction does it all, dense code • Complex instruction, slow execution; pipeline stages often wasted; complex hardware design • Execution cycles: Typically 5-8 cycles per instruction
Types	ARM, RISC-V, MIPS, LoongArch	x86

RISC Architectures & Chip Examples

- ARM
  - **Huawei Kunpeng 920 (server)**
  - Apple M1 (Laptop)
  - Qualcomm Snapdragon (Mobile)
- RISC-V
  - **Alibaba T-Head Xuantie C910 (HPC)**
  - SiFive Intelligence X160 (AI acceleration)
  - ESP32-C6 (IoT)
- MIPS
  - Microchip PIC32 (Industrial control)
- LoongArch
  - **Loongson 3A6000 (Desktop)**



Which ISAs “Win”

- The big winners: x86/x64 (servers) and Arm (phones/embedded)
  - Neither are the cheapest nor the best architectures available...
  - They won because of the legacy ecosystem
- But since our focus is understanding how computers work, we choose learning RISC-V
- Learn to program in assembly language, e.g. RISC-V
  - Best way to understand what compilers do to generate machine code
  - Best way to understand what the CPU hardware does

And the Road To Future Classes

- CS302 Operation Systems
  - OS needs a small amount of assembly for doing things the "high level" language doesn't support
    - Such as accessing special resources
- CS323 Compilers
  - Learn how to build compilers. A compiler goes from source code to assembly language.
- CS301 Embedded System
  - Assembly or a combination of high-level languages and inline assembly are commonly used to achieve efficient execution in resource-constrained environments.
- CS315 Computer Security
  - Exploit code ("shell code") is often in assembly and exploitation often requires understanding the assembly language & calling-convention of the target

zhangjf@sustech.edu.cn

What is RISC-V

- Fifth generation of RISC design from UC Berkeley
- A high-quality, license-free, royalty-free RISC ISA specification
  - Implementers do not pay any royalties
  - Large community of users riscv.org: industry, academia
  - Full software stack
- Appropriate for all levels of computing system, from microcontrollers to supercomputers
  - 32-bit, 64-bit, and 128-bit variants (we're using 32-bit(RV32) in lectures and labs, textbook uses 64-bit)
- Standard maintained by non-profit RISC-V Foundation

The RISC-V Instruction Set

- Used as the example throughout the book
- Developed at UC Berkeley as open ISA
- Now managed by the RISC-V Foundation (riscv.org)
- Typical of many modern ISAs
- Similar ISAs have a large share of embedded core market
  - Applications in consumer electronics, network/storage equipment, cameras, printers, …

Arithmetic Instructions, Operations, Operands

zhangj4@sustech.edu.cn

A Basic Arithmetic Operation

- C

```
a = b + c;  // a statement
```

- Assembly

```
add a, b, c  // an instruction
```

- Operation, operator, operand
- All arithmetic operations have this form
- Example
  - C code: f = (g + h) - (i + j);

RISC-V Instructions/Operations

Category	Instruction	Example
Arithmetic	Add	add x5, x6, x7
	Subtract	sub x5, x6, x7
	Add immediate	addi x5, x6, 20
Data transfer	Load doubleword	ld x5, 40(x6)
	Store doubleword	sd x5, 40(x6)
	Load word	lw x5, 40(x6)
	Load word, unsigned	lwu x5, 40(x6)
	Store word	sw x5, 40(x6)
	Load halfword	lh x5, 40(x6)
	Load halfword, unsigned	lhu x5, 40(x6)
	Store halfword	sh x5, 40(x6)
	Load byte	lb x5, 40(x6)
	Load byte, unsigned	lbu x5, 40(x6)
	Store byte	sb x5, 40(x6)
	Load reserved	lr.d x5, (x6)
	Store conditional	sc.d x7, x5, (x6)
	Load upper immediate	lui x5, 0x12345
Logical	And	and x5, x6, x7
	Inclusive or	or x5, x6, x8
	Exclusive or	xor x5, x6, x9
	And immediate	andi x5, x6, 20
	Inclusive or immediate	ori x5, x6, 20
	Exclusive or immediate	xori x5, x6, 20

Category	Instruction	Example
Shift	Shift left logical	sll x5, x6, x7
	Shift right logical	srl x5, x6, x7
	Shift right arithmetic	sra x5, x6, x7
	Shift left logical immediate	slli x5, x6, 3
	Shift right logical immediate	srli x5, x6, 3
	Shift right arithmetic immediate	srai x5, x6, 3
Conditional branch	Branch if equal	beq x5, x6, 100
	Branch if not equal	bne x5, x6, 100
	Branch if less than	blt x5, x6, 100
	Branch if greater or equal	bge x5, x6, 100
	Branch if less, unsigned	bltu x5, x6, 100
	Branch if greater or equal, unsigned	bgeu x5, x6, 100
Unconditional branch	Jump and link	jal x1, 100
	Jump and link register	jalr x1, 100(x5)

Arithmetic Operations

- Example
  - C code: f = (g + h) - (i + j);
  - Assembly code:

```
add  t0, g,  h        # temp t0 = g + h
add  t1, i,  j        # temp t1 = i + j
sub  f,  t0, t1       # f = t0 - t1
```

- Design Principle 1: Simplicity favors regularity
  - Regularity makes implementation simpler
  - Simplicity enables higher performance at lower cost

Assembly Variables: Registers

- Unlike HLL like C or Java, assembly does not have variables
- Assembly language operands are objects called registers
  - Limited number of special places to hold values, built directly into the hardware
  - 32 registers in RISC-V, named from x0~x31
  - Each RISC-V register is 32 bits wide called a “word” (RV32 variant of RISC-V ISA)
  - Registers have **no type**
  - Operation determines how register contents are interpreted
  - In RISC-V, arithmetic operations primarily use registers as source operands
- Design Principle 2: Smaller is faster
  - registers: only 32 storage locations
  - main memory: millions of storage locations

RISC-V Registers

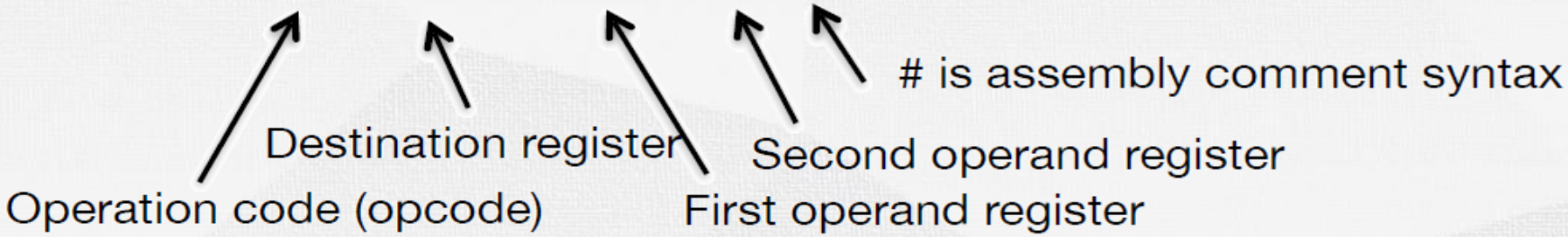
- x0 is special, always holds the value zero and can't be changed

Register	Alternative name	Description
x0	zero	the constant value 0
x1	ra	Return address
x2	sp	Stack pointer
x3	gp	Global pointer
x4	tp	Thread pointer
x5 – x7	t0 – t2	Temporaries
x8	s0/fp	Saved register/Frame pointer
x9	s1	Saved register
x10-17	a0-7	Function arguments/Return values
x18-27	s2-11	Saved registers
x28-31	t3-6	Temporaries

RISC-V Instructions

- Instructions have an opcode and operands

- `E.g., add x1, x2, x3 # x1 = x2 + x3`



- Example
  - C code: f = (g + h) - (i + j); f, …, j in x19, x20, …, x23
  - Compiled RISC-V code:

```
add x5, x20, x21
add x6, x22, x23
sub x19, x5, x6
```

Constant Zero: x0

- Very useful: always holds zero and can never be changed (does not require initialization)
- Ex: Copy a value from one register to another:

```
add x3,x4,x0 (in RISC-V)
same as
f = g (in C)
```

- Whenever a value is produced and we want to throw it away, write to x0

Immediates

- Immediates are used to provide numerical constants
- a++: `addi x3,x3,1`
- a-=3: `addi x3,x3,-3`
- a=0: `addi x3,x0,0`

- Design Principle 3: Make the common case fast
  - Small constants are common
  - Immediate operand avoids loading from memory
- No need of subtract immediate in RISC-V (no *subi* because operation can be done using addi)

Recall: Numeric Representations

- Decimal 35_{10} or 35_{ten}
- Binary 00100011_2 or 00100011_{two}
- Hexadecimal 0x23 or 23_{hex}
  0-15 (decimal) → 0-9, a-f (hex)

Recall: Numeric Representations

- Unsigned Binary Integers
- Given an n-bit number

$$x = x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0$$

- Range: 0 to $+2^n - 1$
- Example
  - $0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 0000\ 1011_2$
    $= 0 + \ldots + 1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0$
    $= 0 + \ldots + 8 + 0 + 2 + 1 = 11_{10}$
- Using 32 bits
  - 0 to +4,294,967,295

Signed Numeric Representations

- 2s-Complement Signed Integers
- Given an n-bit number

$$x = -x_{n-1}2^{n-1} + x_{n-2}2^{n-2} + \cdots + x_1 2^1 + x_0 2^0$$

- Range: -2^{n-1} to $+2^{n-1} - 1$
- Example
  - $1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1111\ 1100_2$
    $= -1\times2^{31} + 1\times2^{30} + \ldots + 1\times2^2 + 0\times2^1 + 0\times2^0$
    $= -2{,}147{,}483{,}648 + 2{,}147{,}483{,}644 = -4_{10}$
- Using 32 bits
  - –2,147,483,648 to +2,147,483,647

Signed Numeric Representations

- Signed Negation
- Complement and add 1
  - Complement means 1 → 0, 0 → 1

$$x + \bar{x} = 1111\ldots111_2 = -1$$

$$\bar{x} + 1 = -x$$

- Example: negate +2
  - +2 = 0000 0000 … 0010_2
  - –2 = 1111 1111 … 1101_2 + 1
    = 1111 1111 … 1110_2

Sign Extension

- addi immediate are limited to 12 bits (We'll see more details in future lecture)
- In RISC-V immediates are "sign extended" to 32 bits
  - So the upper bits are the same as the signed bit (bit[11])
- Examples: 12-bit to 32-bit sign extension
  - $+1_{10}$ $0000_0000_0001_2$ → 0000_0000_0000_000_0000_0000_0000_0001
  - -3_{10} $1111_1111_1101_2$ → 1111_1111_1111_1111_1111_1111_1111_1101
- Binary negative value
- -3 is obtained by finding two's complement of 3
  - abs(-3) →3 → 0000_0000_0011
  - one's complement → 1111_1111_1100
  - add 1 → 1111_1111_1101

Data Transfer and Memory Access Mechanisms

zhangj4@sustech.edu.cn

Data Transfer Operations

- Registers vs. Memory
  - Arithmetic operations can only be performed on registers
  - Thus, the only memory actions are loads & stores
- Given that
  - Registers: 32 words (128 Bytes)
  - Memory (DRAM): Billions of bytes (2 GB to 16 GB on laptop)
- How much faster are registers than DRAM??
- About 100-500 times faster!
  - in terms of latency of one access

Byte-Addressable Memory

- RISC-V is byte-addressed, each byte has unique address
- 1 word = 32-bit = 4 bytes, so word address increments by 4
  - the address of memory word 2 is 2 × 4 = 8
  - the address of memory word 10 is 10 × 4 = 40 (0x28)

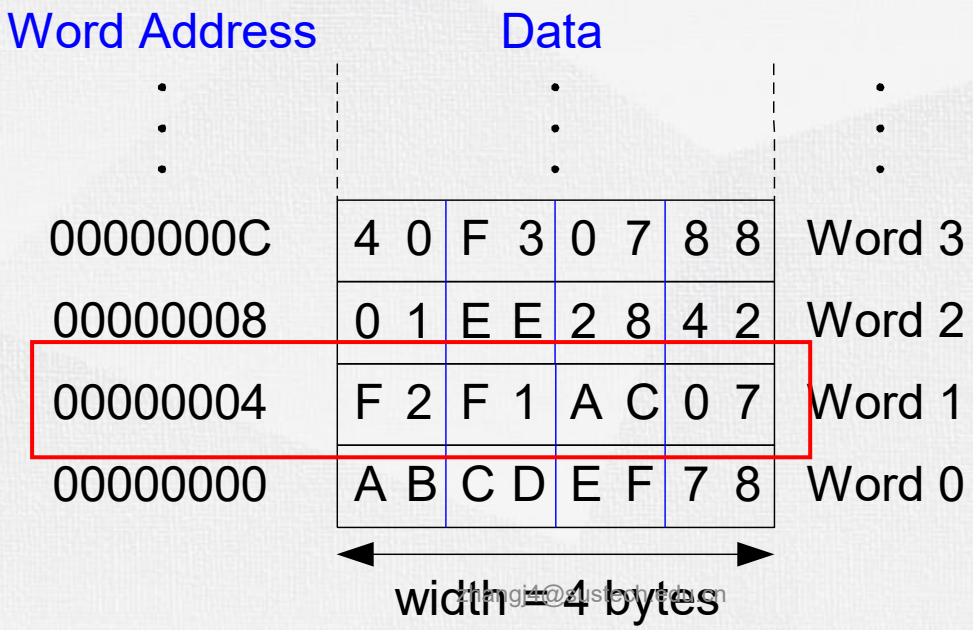
Word Address	Data	
⋮	⋮	⋮
0000000C	4 0 F 3 0 7 8 8	Word 3
00000008	0 1 E E 2 8 4 2	Word 2
00000004	F 2 F 1 A C 0 7	Word 1
00000000	A B C D E F 7 8	Word 0

width = 4 bytes

0xf	40		word3
0xe	f3		
0xd	07		
0xc	88	0xc	
0xb	01		word2
0xa	EE		
0x9	28		
0x8	42	0x8	
0x7	F2		word1
0x6	F1		
0x5	AC		
0x4	07	0x4	
0x3	AB		word0
0x2	CD		
0x1	EF		
0x0	78	0x0	

Memory Read - Load

- Example: Load a word of data at memory address 4 into x3.
- RISC-V assembly code
  - `lw  x3,4(x0)  # read word at address x0 + 4 into x3`
    `# x0: base address, 4: offset`
- x3 holds the value 0xF2F1AC07 after load (little endian)



chengjh@sustech.edu.cn

Memory Read - Load

- C code

```
int A[100];
g = h + A[8];
```

  - Assume: x13 holds base register (pointer to A[0])
  - offset in bytes = 8*4 = 32
  - Offset must be a constant known at assembly time

- Using Load Word (**lw**) in RISC-V:

```
lw  x10,32(x13)  # x10 = 0x12345678
add x11,x12,x10  # g = h + A[8]
```

Word Addr.	Data				
…	…				…
base + 32	12	34	56	78	A[8]
…					
base + 8	C1	C2	01	31	A[2]
base + 4	FA	B3	10	45	A[1]
base addr.	D0	32	12	45	A[0]
…	…				…

Memory Write - Store

- C code

```
int A[100];
A[10] = h + A[8];
```

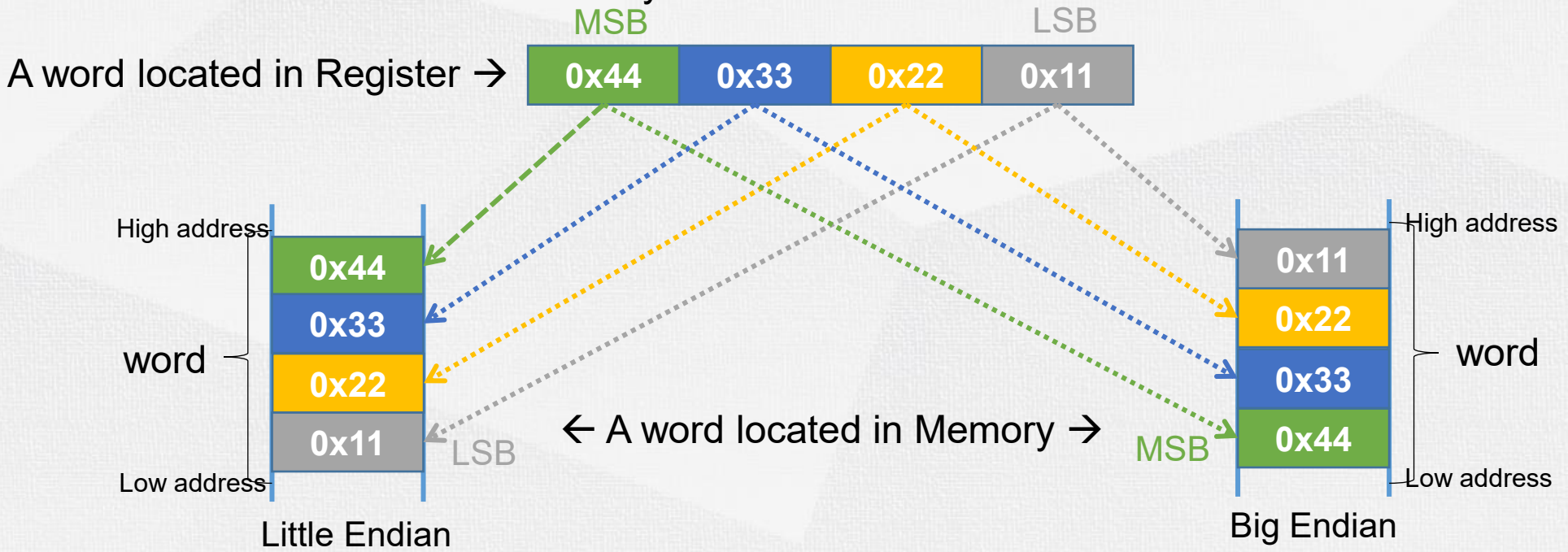
  - Assume: x13 holds base register (pointer to A[0]), x12 holds h
  - Note: 32, 40 is offset in bytes
  - Offset must be a constant known at assembly time
- Using Store Word (**sw**) in RISC-V:

```
lw  x10,32(x13)  # reg x10 gets A[8]
add x11,x12,x10  # g = h + A[8]
sw  x11,40(x13)  # A[10] = h + A[8]
```

- x13+32 and x13+40 must be multiples of 4 to maintain alignment

Endianness - Little Endian vs Big Endian

- Endianness: byte ordering within a word
  - Little-endian (e.g. RISC-V)
    - LSB of a word is at least memory address
  - Big-endian (e.g. MIPS)
    - MSB of a word is at least memory address

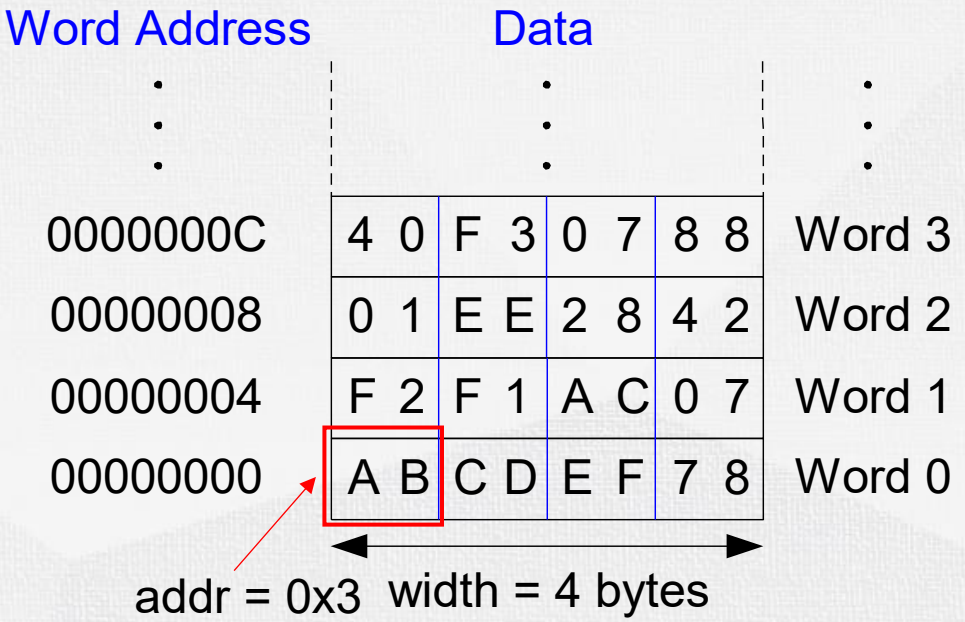


LSB is at least address!

MSB is at least address!

Byte Data Transfer

- In addition to word data transfers (lw, sw), RISC-V has byte data transfers:
  - load byte: lb
  - store byte: sb
- Same format as lw, sw
- `lb x10,3(x0)`
  - Byte stored at 0x3 is 0xAB
  - Sign extension → x10 = 0xFFFFFFAB



- RISC-V also has "unsigned byte" loads (lbu) which zero extend to fill register.
- `lbu x11,3(x0)`
  - Zero extension → x11 = 0x000000AB

Endianness & Byte Transfer Example

- Example: For the following RISC-V code, what's the final value in x12?

```
addi x11,x0,0x3f5
sw x11,0(x5)
lb x12,1(x5)
```

Answer	x12
A	0x5
B	0xf
C	0x3
D	0xffffffff

- for this example
  - byte[0] = 0xf5
  - byte[1] = 0x03
  - byte[2] = 0x00
  - byte[3] = 0x00

Endianness & Byte Transfer Example 2

- Example: For the following RISC-V code, What's the final value in x12?

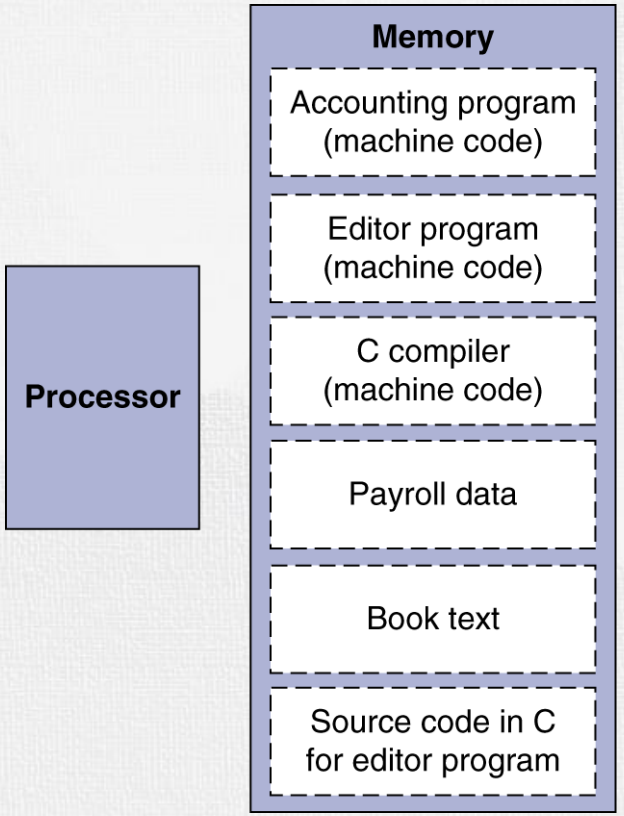
```
addi x11,x0,-512
sw x11,0(x5)
lb x12,1(x5)
```

Answer	x12
A	0xe
B	0xfe
C	0x0
D	0xfffffffe

- The immediate got sign extended...
  - So $-512_{10} = -2^9$ = 0xfffffe00 got written
- Then load byte is called
  - So it will load byte[1], which is 0xfe
- But load byte sign extends too...
  - So what gets loaded into the register is 0xfffffffe
- If we did lbu we'd instead get 0xfe

Stored Program Computers

- Instructions represented in binary, just like data
- Instructions and data stored in memory
- Programs can operate on programs
  - e.g., compilers, linkers, …
- Binary compatibility allows compiled programs to work on different computers
  - Standardized ISAs



Logic and Flow Control Instructions

zhangj4@sustech.edu.cn

Logical Operations

- Useful for extracting and inserting groups of bits in a word

Operation	C	Java	RISC-V
Shift left logical	<<	<<	sll
Shift right	>>	>>	srl/sra
Bitwise AND	&	&	and
Bitwise OR	|	|	or
Bitwise XOR	~	~	xor

- Shift left logical
  - Shift left and fill with 0 bits
  - `slli` by *i* bits multiplies by 2^i
- Shift right logical
  - Shift right and fill with 0 bits
  - `srli` by *i* bits divides by 2^i (unsigned only)
- Shift right arithmetic
  - Shift right and fill with sign bits

Logical Instructions

- AND: clear some bits
  - `and x9,x10,x11`
- OR: set some bits
  - `or x9,x10,x11`
- XOR: toggle some bits
  - `xor x9,x10,x12`
- How about NOT?
  - Can be implemented with XOR
  - `xori x15,x14,-1`

```
      0x35   0 0 1 1 0 1 0 1
AND   0x0F   0 0 0 0 1 1 1 1
      0x05   0 0 0 0 0 1 0 1

      0x04   0 0 0 0 0 1 0 0
OR    0x30   0 0 1 1 0 0 0 0
      0x34   0 0 1 1 0 1 0 0

      0x44   0 1 0 0 0 1 0 0
XOR   0x06   0 0 0 0 0 1 1 0
      0x34   0 1 0 0 0 0 1 0
```

Data Transfer with Variable Indexing

- C code

```
int A[100];        /* A[0] address is in x13 */
int i;             /* i in x14 */
...
g = h + A[i]; /* h = x12, g = x11, tmp = x15 */
```

- Using Load Word (lw) in RISC-V with pointer arithmetic:

```
slli x15,x14,2    # Multiply i by 4 for ints
add x15,x15,x13   # A + 4 * i
lw x10,0(x15)
add x11,x12,x10
```

Conditional Operations

- Branch to a labeled instruction if a condition is true
  - Otherwise, continue sequentially
- Conditional branch
  - *beq rs1, rs2, L1*

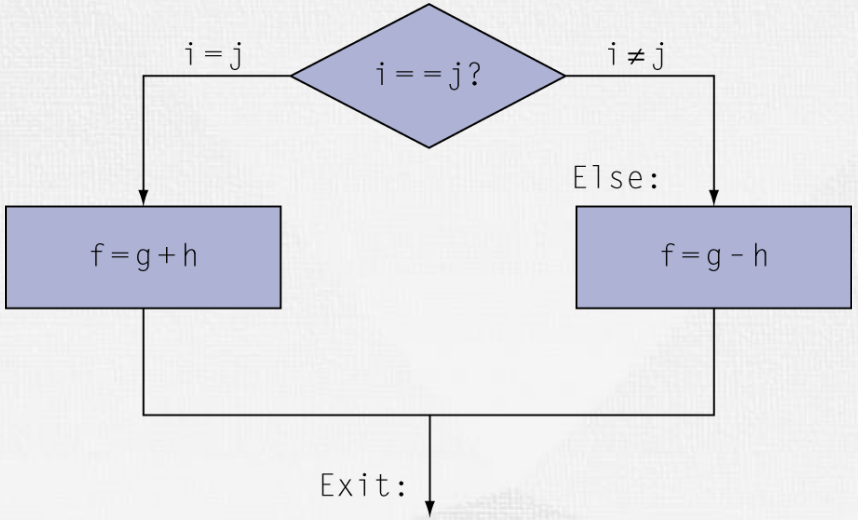
    if (rs1 == rs2) branch to instruction labeled L1;
  - *bne rs1, rs2, L1*
    - if (rs1 != rs2) branch to instruction labeled L1;
- Unconditional branch
  - *beq x0, x0, L1*
    - unconditional jump to instruction labeled L1

Compiling If Statements

- C code

```
if (i==j) f = g+h;
else f = g-h;
```

  - i and j are in x22 and x23,
  - f,g and h are in x19, x20 and x21



- Compiled RISC-V code:

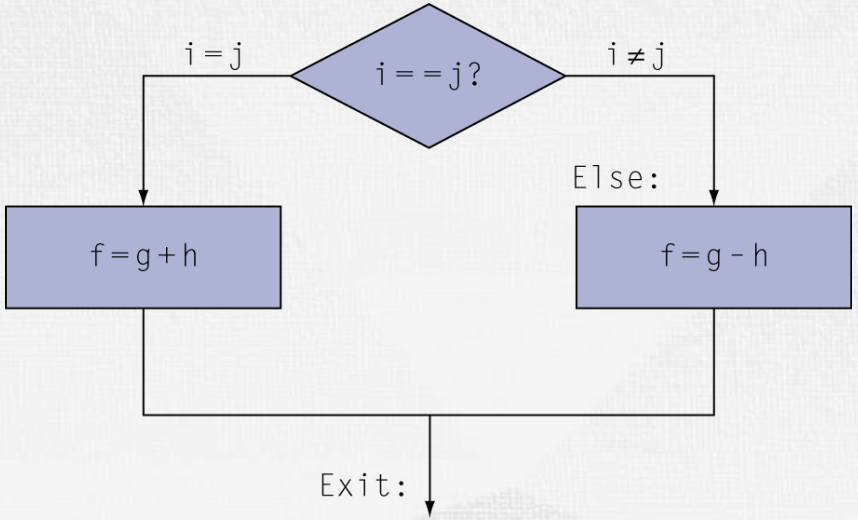
```
        bne x22, x23, Else  # go to Else if i ≠ j
        add x19, x20, x21   # f=g+h, skipped if i ≠ j

Else:   sub x19, x20, x21   # f=g-h, skipped if i = j
```

Compiling If Statements

- C code

```
if (i==j) f = g+h;
else f = g-h;
```

  - i and j are in x22 and x23,
  - f,g and h are in x19, x20 and x21



- Compiled RISC-V code:

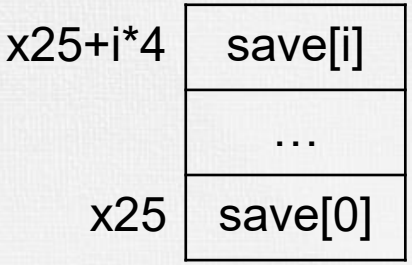
```
        bne x22, x23, Else  # go to Else if i ≠ j
        add x19, x20, x21   # f=g+h, skipped if i ≠ j
        beq x0, x0, Exit    # unconditional go to Exit
Else:   sub x19, x20, x21   # f=g-h, skipped if i = j
Exit:
```

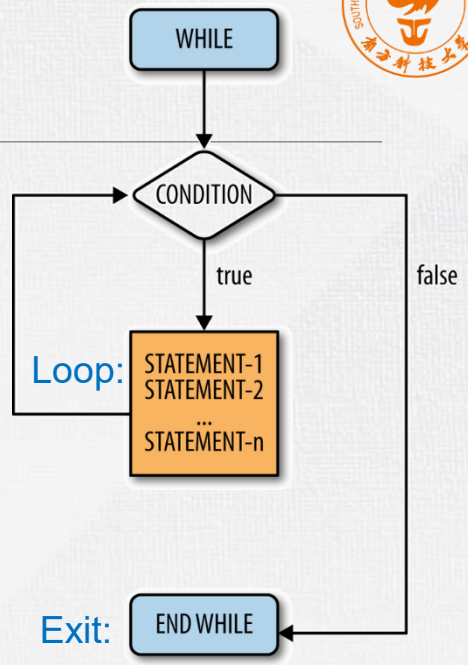
Compiling Loop Statements

- C code:

```
while (save[i] == k)
    i += 1;
```

  - i in x22, k in x24, address of save in x25





- Compiled RISC-V code:

```
Loop: slli   x10, x22, 2    # Temp reg x10 = i * 4
      add    x10, x10, x25  # x10 = address of save[i]
      lw     x9, 0(x10)     # Temp reg x9 = save[i]
      bne    x9, x24, Exit  # go to Exit if save[i]≠k
      addi   x22, x22, 1    # i = i + 1
      j      Loop           # go to Loop
Exit:
```

More Conditional Operations

- Signed comparison
  - `blt rs1, rs2, L1`
    - if (rs1 < rs2) branch to instruction labeled L1
  - `bge rs1, rs2, L1`
    - if (rs1 >= rs2) branch to instruction labeled L1
  - Example, C to RISC-V
    - `if (a > b) a += 1;`
    - a in x22, b in x23

```
    bge  x23, x22, Exit # signed comparison
    addi x22, x22, 1
Exit:
```

- Unsigned comparison
  - `bltu, bgeu`

What if we need more instructions?

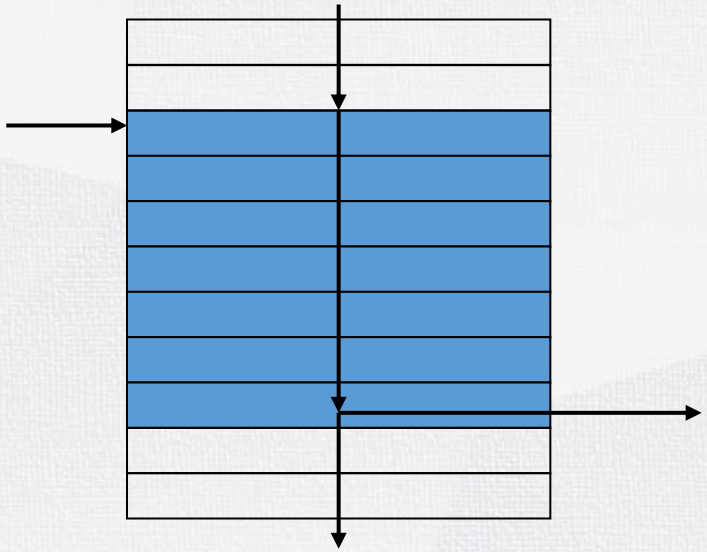
- RISC-V doesn’t have "branch if greater than” or “branch if less than or equal”
- Instead you can reverse the arguments, as:
  - A > B is equivalent to B < A
  - A <= B is equivalent to B >= A
- The assembler defines pseudo-instructions for your convenience:

```
bgt x2, x3, foo (pseudo)
blt x3, x2, foo (basic)
```

will become

Basic Blocks

- A basic block is a sequence of instructions with
  - No embedded branches (except at end)
  - No branch targets (except at beginning)



  - A compiler identifies basic blocks for optimization
  - An advanced processor can accelerate execution of basic blocks

C to RISC-V Example

```
int A[20];
int sum = 0;
for (int i=0; i<20; i++)
        sum += A[i];
```

```
    # Assume x8 holds pointer to A
    # Assign x10=sum, x11=i
1   add x10, x0, x0        # sum=0
2   add x11, x0, x0        # i=0
3   addi x12,x0,20         # x12=20
4   loop:
    bge x11, x12, exit
5   slli x13, x11, 2       # i * 4
6   add x13, x13, x8       # A + i
7   lw x13, 0(x13)         # *(A + i)
8   add x10, x10, x13      # increment sum
9   addi x11, x11, 1       # i++
10  beq x0, x0, loop       # iterate
11  exit:
```

Loop has 7 instructions

C to RISC-V Example Optimized

```
int A[20];
int sum = 0;
for (int i=0; i<20; i++)
        sum += A[i];
```

```
    # Assume x8 holds base address of A
    # Assign x10=sum, x11=i*4
1   add x10, x0, x0   # sum=0
2   add x11, x0, x0   # i=0
3   addi x12,x0,80    # x12=20*4
4   loop:
    bge x11, x12, exit
5   add x13, x11, x8        # A + i
6   lw x13, 0(x13)          # *(A + i)
7   add x10, x10, x13       # increment sum
8   addi x11, x11, 4        # i++
9   beq x0, x0, loop        # iterate
10  exit:
```

Loop now has 6 instructions

C to RISC-V Example Optimum

```
int A[20];
int sum = 0;
for (int i=0; i<20; i++)
        sum += A[i];
```

Loop now has 4 instructions

- Directly increment ptr into A array
- And only 1 branch/jump rather than two
  - Because first time through is always true so can move check to the end
  - The compiler will often do this automatically for optimization

```
# Assume x8 holds base address of A
# Assign x10=sum
# Assume x11 holds ptr to next A
add x10, x0, x0        # sum=0
add x11, x0, x8        # Initial address
addi x12, x8, 80       # End address
loop:
lw x13, 0(x11)
add x10, x10, x13
addi x11, x11, 4
blt x11, x12, loop
exit:
```

Summary of Design Principles

1. Simplicity favors regularity
   - Keep all instructions the same size.
2. Smaller is faster
   - Register vs memory
   - Number of registers is small
3. Make the common case fast
   - Immediate operand
4. Good design demands good compromises
   - Keep formats as similar as possible

Summary

1. Instruction set architecture (ISA) specifies the set of commands (instructions) a computer can execute
2. Hardware registers provide a few very fast variables for instructions to operate on
3. RISC-V ISA requires software to break complex operations into a string of simple instructions, but enables faster, simple hardware
4. Assembly code is human-readable version of computer's native machine code, converted to binary by an assembler